

预备知识：

- 一、计算机中的数据表示、二进制
- 二、进位计数制：十进制、二进制、八进制、
十六进制数的表示
- 三、不同进制数之间的转换
- 四、数的机器码表示
- 五、字符的编码表示

1、计算机中，数据都是采用**二进制编码**的形式表示的， 且**按字节分配存储**。

二进制：一种只有0和1两个数码、**逢2进1**的进位计数制。

(1) **一个字节**：8个bit（位）

- ◆ 0-1编码范围：00000000~11111111

- ◆ 可表示的整数范围：

 - ▣ 有符号数：(-128~127)₁₀

表示是十进制数

 - ▣ 无符号数：(0~255)₁₀

(2) **二个字节**：16个bit

- ◆ 0-1编码范围：00000000~0~11111111~1（16个0或1）

- ◆ 可表示的整数范围：

 - ▣ 有符号数：-32768~32767

 - ▣ 无符号数：0~65535

(3) **四个字节**：32个bit

- ◆ 0-1编码范围：0000~0~1111~1 (32个0或1)
- ◆ 可表示的整数范围：
 - ▣ 有符号数：-2147483648 ~ 2147483647
 - ▣ 无符号数：0 ~ 4294967295

2、十进制、二进制、八进制、十六进制数之间的转换

(1) 十进制转二进制

- ◆ 整数：**除基取余**
- ◆ 小数：**乘基取整**

(2) 二进制转十进制：**按权展开**

(3) 十进制转N进制

- ◆ 整数：**除基取余**
- ◆ 小数：**乘基取整**

(4) N进制转十进制：**按权展开**

3、数据的编码表示

(1) 整数：原码、反码、补码

计算机中的整数都是采用补码表示的：

- 正数的补码就是其原码
- 负数的补码是符号位为1，数值位在原码的基础上取反，然后整体加1（或在绝对值（正数）原码的基础上，整体取反加1）

(2) 浮点数：IEEE 754标准

- ◆ 符号位：1位，最高位
- ◆ 阶码：8位（float）或11位（double）
- ◆ 位数：23位（float）或52位（double）

4、字符数据的表示

(1) 字符的图形是给人看的，计算机中存储的是**字符的编码**。

字符编码本质上是一个**非负整数**。

(2) ASCII码

- ◆ **128个**英文字符，编码：**0~127**，**用1字节表示**
- ◆ 记住**数字**、**A~Z**、**a~z**等一些关键字符的编码
记住第一个即可，后面的可以推导出来。
- ◆ **标准ASCII码**：8位编码中的**最高位为0**
- ◆ **扩展ASCII码**：8位编码中的**最高位为1**

(3) 中文采用**2字节编码**

(4) 国际字符编码（了解即可）

自学以下内容（2-4 学时）

一、数和进位计数制

数目表示事物的个数，表示**数目**的符号称为**数字**，简称为**数**。

同样大小的**数（数目）**，在不同的**进位制**下有不同的表示形式，形成**数制的概念**。

- ◆ **数制**是关于计数的法则
- ◆ 根据进位**基数**的不同，计算机领域常用的进位计数制有：
十进制、二进制、八进制、十六进制等。

1. 十进制数

日常生活中通常使用十进制数。

十进制中，**数字符号**（简称为**数符**）有 **0、1、2、3、4、5、6、7、8、9** 十个，**运算规则是逢十进一**。

十进制数是由 **0~9** 组成的数字串，如 **123456**。为与其它进制数区分，通常也会写成：**(123456)**

10

表示这是一个十进制数

- ◆ 一位的十进制数：0、1、2、3、4、5、6、7、8、9
- ◆ 两位的十进制数：00、01、10、11、.....、99
- ◆ 三位的十进制数：000、001、...、100、101、.....、999
- ◆

2. 二进制数

二进制中，**数符**只有**0**和**1**两个，**运算规则是逢2进1**。

二进制数是由**0**和**1**组成的数字串，通常表示为：

如：(10110110)₂



表示这是一个二进制数

- ◆ 一位的二进制数：0、1
- ◆ 两位的二进制数：00、01、10、11
- ◆ 三位的二进制数：000、001、010...、111
- ◆

二进制数的运算

(1) 一位的二进制数 (0或1) 对应十进制的0或1:

$$(0)_2 \sim (0)_{10}$$

$$(1)_2 \sim (1)_{10}$$

(2) 二进制加法:

$$0+0=0$$

$$0+1=1+0=1$$

$$1+1=10$$

$$\begin{array}{r} 1 \\ +1 \\ \hline 10 \end{array}$$

逢二进一：向高位进位1

多位二进制数相加

求 $(10110111)_2 + (01011011)_2$

$$\begin{array}{r} 10110111 \\ + 01011011 \\ \hline \text{进位 } 1111111 \\ 100010010 \end{array}$$

(3) 二进制减法:

$$0-0=0$$

$$1-0=1$$

$$1-1=0$$

$$10-1=1$$

$$\begin{array}{r} 10 \\ - 1 \\ \hline 1 \end{array}$$

借一作二：向高位借1

多位二进制数相减

$$\text{求}(10110111)_2-(01011011)_2$$

$$\begin{array}{r} \text{借位} \quad 1011000 \\ \quad 10110111 \\ - \quad 01011011 \\ \hline \quad 01011100 \end{array}$$

$$\begin{aligned} & (10110111)_2 - (01011011)_2 \\ & = (01011100)_2 \end{aligned}$$

(4) 二进制乘、除：其它课程

二进制数和十进制数的对应关系

- ◆ 一位的二进制数 $(0)_2$ 和 $(1)_2$ 分别等于十进制的0和1;
- ◆ 两位的二进制数 $(00)_2$ 、 $(01)_2$ 分别等于十进制的0和1;
- ◆ 两位的二进制数 $(10)_2$ 、 $(11)_2$ 和十进制的多少对应?

$$(1)_2 + (1)_2 = (10)_2 == (2)_{10}$$

$$(10)_2 + (1)_2 = (11)_2 == (3)_{10}$$

二进制加法：逢2进1

- ◆ 三位的二进制数和十进制数的对应关系

十进制	0	1	2	3	4	5	6	7
二进制	000	001	010	011	100	101	110	111

四位的二进制数和十进制数的对应关系

十进制	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
二进制	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

n位的二进制数可以表示十进制的 **$0 \sim 2^n - 1$** ：
 $\underbrace{00 \dots 0}_{n \text{ 个 } 0}, \dots, \underbrace{11 \dots 1}_{n \text{ 个 } 1}$

3. 八进制数

八进制中，**数符**有**0、1、2、3、4、5、6、7**八个，**运算规则是逢8进1**。

八进制数是由 **0~7** 组成的数字串，通常表示为：

如：(136)₈

表示这是一个八进制数

- ◆ 一位的八进制数：0、1、2、3、4、5、6、7
- ◆ 两位的八进制数：00、01、02、...、10、11、...、77
- ◆ 三位的八进制数：000、001、...、777
- ◆

八进制数的运算

(1) 一位的八进制数 (0~7) 对应十进制的0~7:

$$(0)_8 \sim (0)_{10}、\dots\dots、(7)_8 \sim (7)_{10}$$

(2) 八进制加法:

$$7 + 1 = 10$$

$$\begin{array}{r} 7 \\ + 1 \\ \hline 10 \end{array}$$

逢八进一：向高位进位1

(3) 八进制减法:

$$10 - 1 = 7$$

$$\begin{array}{r} 10 \\ - 1 \\ \hline 7 \end{array}$$

借一作八：向高位借1

八进制数和十进制数的对应关系

- ◆ 一位的八进制数 $(0)_8 \sim (7)_8$ 分别等于十进制的 $0 \sim 7$;
- ◆ 两位的八进制数 $(00)_8 \sim (07)_8$ 分别等于十进制的 $0 \sim 7$;
- ◆ 两位的八进制数 $(10)_8$ 、 $(11)_8$ 和十进制的多少相对应?

$$(07)_8 + (1)_8 = (10)_8 == (8)_{10}$$

$$(10)_8 + (1)_8 = (11)_8 == (9)_{10}$$

八进制加法：逢8进1

- ◆ 其它依次类推
- ◆ n 位八进制数可以表示十进制的 $0 \sim 8^n - 1$ ：
 $\underbrace{00 \dots 0}_{n \uparrow 0}, \dots, \underbrace{77 \dots 7}_{n \uparrow 7}$

4. 十六进制数

(A~F也可以采用小写英文字母a~f)

十六进制中，**数符**有 **0、1、2、3、4、5、6、7、8、9、A、B、C、D、E、F** 十六个，**运算规则是逢16进1**。

十六进制数是由 **0~9** 和 **A~F** 组成的数字串，通常表示为：

如：(2D5B)₁₆



表示这是一个十六进制数

- ◆ 一位的十六进制数：0、1、2、3、4、5、6、7、8、9、
A、B、C、D、E、F
- ◆ 两位的十六进制数：00、01、...、**0F**、**10**、11、...、FF
- ◆ 三位的十六进制数：000、001、...、FFF
- ◆

十六进制数的运算

(1) 一位的十六进制数 **0~9** 对应十进制的0~9。

$$(0)_{16} \sim (0)_{10}、 \dots\dots、 (9)_{16} \sim (9)_{10}$$

一位的十六进制数 **A~F** 对应十进制的**10~15**。

$$(A)_{16} \sim (10)_{10}$$

$$(B)_{16} \sim (11)_{10}$$

$$(C)_{16} \sim (12)_{10}$$

$$(D)_{16} \sim (13)_{10}$$

$$(E)_{16} \sim (14)_{10}$$

$$(F)_{16} \sim (15)_{10}$$

(2) 十六进制加法:

$$F + 1 = 10$$

$$\begin{array}{r} F \\ + 1 \\ \hline 10 \end{array}$$

逢十六进一：向高位进位1

(3) 十六进制减法:

$$10 - 1 = F$$

$$\begin{array}{r} 10 \\ - 1 \\ \hline F \end{array}$$

$$(ABCD)_{16} + (2d5b)_{16} = (D928)_{16}$$

$$(ABCD)_{16} - (2d5b)_{16} = (7E72)_{16}$$

借一作十六：向高位借1

十六进制数和十进制数的对应关系

- ◆ 一位的十六进制数 $(0)_{16} \sim (F)_{16}$ 分别等于十进制的 $0 \sim 15$;
- ◆ 二位的十六进制数 $(00)_{16} \sim (0F)_{16}$ 分别等于十进制的 $0 \sim 15$;
- ◆ 二位的十六进制数 $(10)_{16}$ 、 $(11)_{16}$ 和十进制的多少相对应?

$$(10)_{16} == (16)_{10}$$

$$(11)_{16} == (17)_{10}$$

- ◆ 其它依次类推
- ◆ n 位十六进制数可以表示十进制的 $0 \sim 16^n - 1$:

$$\underbrace{00 \dots 0}_{n \text{ 个 } 0}, \dots, \underbrace{FF \dots F}_{n \text{ 个 } 7}$$

十进制数0~15的二进制、八进制、十六进制表示

十进制	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
二进制	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
八进制	00	01	02	03	04	05	06	07	10	11	12	13	14	15	16	17
十六进制	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

问：10代表的数目是多少？（用十进制的大小回答）

5、N进制数的位数与其可表示的数的范围

位数越多，可表示的数就越多：

- ◆ 1位的十进制数只能表示**0~9**
- ◆ 2位的十进制数能表示**0~99**
- ◆ ...
- ◆ n位的十进制数能表示 **$0 \sim 10^n - 1$**

同理，

- ◆ 1位的二进制数只能表示**0~1**
- ◆ 2位的二进制数只能表示**0、1、2、3**
- ◆ ...
- ◆ n位的二进制数能表示 **$0 \sim 2^n - 1$**

**注：以十进制理解
可表示的数**

位数越多，可表示的数就越多。具体应用时，需要根据数据对象的大小确定数据的位数（**数据长度**），位数（**数据长度**）不够，就表示不了要表示的数。如，

- ◆ **128**至少要用**3位**的十进制数表示；
- ◆ 如果要表示的数不超过128（**0~127**），可用**7位**的**二进制数**表示（ $2^7-1=127$ ）；
- ◆ 如果要表示的数**超过128**，则至少要用**8位**或更多位的二进制数表示。

注：以十进制理解可表示的数

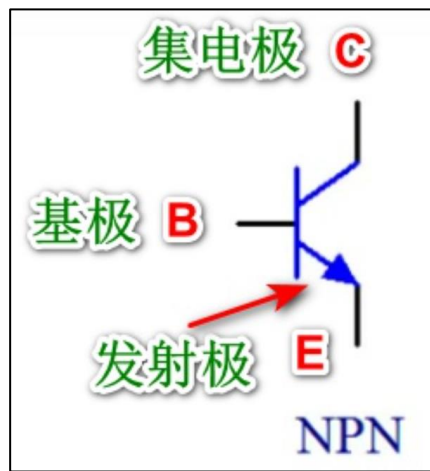
问题：1个字节能表示多大的数？

计算机内部的所有数据（包括代码）都用二进制表示。

“数据存放在内存的字节单元中” 如何理解？

（以整数为例）

一个字节包含8个bit，每个bit可以表示一位二进制数0或1。



三极管

（在计算机内部，**每个bit对应一个最小的电子部件**，它会有两个相反状态——如三极管，基极B加电压，发射极E就有输出，否则就无输出，这样可以根据E端有无输出定义出三极管的两个不同状态：**通或不通**，令其中一个状态代表1，另一个状态代表0，就可以用它表示出数0或1）。

一个字节包含**8个bit**，每个bit可以表示一位的0或1，
则一个字节可以表示的数有**256个**（**从0开始不断 +1**）：

00000000 00000001 00000010 00000011

00000100 00000101 00000110 00000111

00001000 00001001 00001010 00001011

.....

11111100 11111101 11111110 11111111

—— 分别对应十进制数的**0~255**。

所以**一个字节可以存储一个不超过255大小的数**。

◆ 超过255的数怎么存储（在计算机中表示）？

一个字节显然不够，就用多个字节：

2个连续的字节合在一起有**16位**，可以表示0~65535

(在16位范围内从0开始不断 +1)

字节1 字节2

00000000 00000000

┌

11111111 11111111

共可表65536个数

4个连续的字节有**32位**，可以表示0~ $2^{32}-1$

以此类推，用越多的字节就可表示越大的数据范围

二、不同进位制数之间的转换

基：N进制的基数是**N**。

1. 十进制整数转换为N进制整数：除基取余法

以十进制整数转换二进制整数为例：**除2取余法**

用除2取余法求十进制整数 189 的二进制表示。

解

2 189	
2 94	余 1
2 47	余 0
2 23	余 1
2 11	余 1
2 5	余 1
2 2	余 1
2 1	余 0
0	余 1 (最高位)

除到商为0为止

从下往上（反序）将所得的余数
梳理成一行，即得该十进制整数的
二进制表示。

$(189)_{10} = (10111101)_2$

←
高位

低位

2) 十进制纯小数转换N进制小数：乘基取整法

以二进制为例：乘2取整法

如：将 $(0.6875)_{10}$ 转换为二进制小数表示

$$0.6875 \times 2 = 1.375 \quad \begin{matrix} 1 \\ \downarrow \end{matrix} \quad \begin{matrix} \text{高位} \\ \text{低位} \end{matrix}$$

$$0.375 \times 2 = 0.75 \quad 0$$

$$0.75 \times 2 = 1.5 \quad 1$$

$$0.5 \times 2 = 1 \quad 1$$

从上往下（正序） 将所得的整数梳理成一行，即得该十进制纯小数的二进制表示。

$$(0.6875)_{10} = (0.1011)_2$$

高 低

3) 带有整数部的十进制小数转换成二进制数形式

整数部和小数部分别转换，然后合在一起即可。

如： $(189.6875)_{10} = (10111101.1011)_2$

整数部

小数部

2. N进制数转换为十进制数

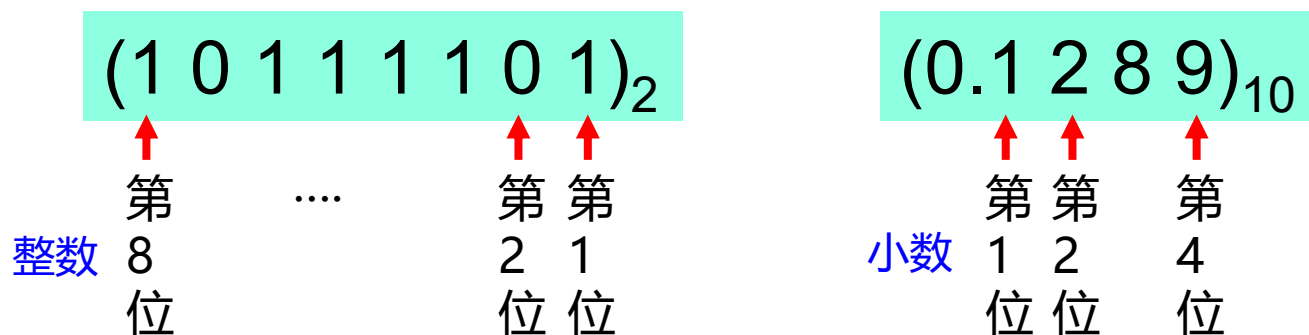
- 类似于十进制的“**个、十、百、千**”位等，N进制的数中也有**高位、低位**，一般低位在右，高位在左。



位号:

整数部: 从右 (最低位) 至左 (最高位) 依次称为第1位、第2位、...、第n位

小数部: 从左至右依次称为第1位、第2位、...、第m位



■ 位的权：不同的位赋以不同的权值

整数部：第 i 位的权为 N^{i-1}

小数部：第 j 位的权为 N^{-j}

	N^4	N^3	N^2	N^1	N^0	.	N^{-1}	N^{-2}	N^{-3}	N^{-4}	
.....	第5位	第4位	第3位	第2位	第1位	小数点	第1位	第2位	第3位	第4位	

◆ N进制数转换为十进制数：**按权展开，乘权求和**

即：**各位的数符 × 该位的权，然后加在一起**

(以二进制为例)

二进制下各位的权值：

	2^4	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}	2^{-4}	
.....	第5位	第4位	第3位	第2位	第1位	小数点	第1位	第2位	第3位	第4位	

◆ 二进制数转换为十进制数：**按权展开，乘权求和**

$$\begin{aligned} \text{如: } (10111101.1101)_2 &= 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + \\ &\quad 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + \\ &\quad 0 \times 2^{-3} + 1 \times 2^{-4} = (189.8125)_{10} \end{aligned}$$

3. 二进制整数转换为八进制整数: 3位分组法

三位二进制数恰好可以表示**0~7**，对应八进制的8个数码。

对二进制整数，**从最低位起，每3位分成1组**，最左边不足3位时前面加0补足3位，然后**把每3位二进制数用一个对应的八进制数符表示**即可。

如：

$$(10111101)_2 = (\underline{010} \ \underline{111} \ \underline{101})_2 = (275)_8$$



前面加0补足3位

4. 八进制整数转换为二进制整数：一变三

对八进制整数，将**每一位的八进制数符用相应大小的3位二进制数**表示即可。

如：

$$\begin{aligned}(171777)_8 &= (\underline{001} \ \underline{111} \ \underline{001} \ \underline{111} \ \underline{111} \ \underline{111})_2 \\ &= (1111 \ \underline{00} 1111111111)_2\end{aligned}$$

5. 二进制整数转换为十六进制整数： **4位分组法**

四位二进制数可以表示**0~15**，对应十六进制的16个数符。

对二进制整数，从最低位起，**每4位分成1组**，最左边不足4位时前面加0补足4位，然后**把每4位二进制数用一个对应大小的十六进制数符**表示即可。

$$\begin{aligned}\text{如：} & \quad (10111101)_2 \\ & = (\underline{1011} \ \underline{1101})_2 \\ & = (\text{BD})_{16}\end{aligned}$$

6. 十六进制整数转换为二进制整数：一变四

对十六进制整数，将每一个十六进制数符用对应大小的**4位二进制数**表示即可。

如：

$$\begin{aligned} (ABCD)_2 &= (\underline{1010} \ \underline{1011} \ \underline{1100} \ \underline{1101})_2 \\ &= (1010101111001101)_2 \end{aligned}$$

- 数有正负，数的大小表示出来了，**符号**怎么办？

二进制数的正负表示：引入**符号位**

对一个n位的二进制数，令**最高一位为符号位**，其余的n-1位为数值位。

符号位：**0**表示这个数是**正数**，**1**表示是**负数**。

如：引入符号位后，四位数 **0100** 表示+4，**1100**表示-4

八位数 **00001111** 表示+15，**10001111**表示-15

注：由于引入**符号位**，数值位只有**n-1位**，所以这时候总共n位的二进制数可表示的数据范围变为： **$-(2^{n-1}-1) \sim +2^{n-1}-1$** 。

可表示的数的总个数没变，还是 2^n 个，但分成了正、负两部分。

- ◆ **带符号数**：带符号位的数称为**带符号数**，简称**符号数**；
- ◆ **无符号数**：不带符号位的数称为**无符号数**。
 - 无符号数中**所有位都作为数值位使用**，其值都被视为**非负数**

如：四位数**0100**：带符号数表示+4，无符号数也表示+4

1100：带符号数表示-4，无符号数则表示+12

八位数**00001111**：带符号数表示+15，无符号数也表示+15

10001111：带符号数表示-15，无符号数表示+143

注：**无符号数**由于**所有位都作为数值位使用**，所以可表示的整数范围是 **$0 \sim 2^n - 1$** ，比带符号数可表示的非负整数范围大一倍。但能够表示的**数值个数是一样的**，都为 **2^n 个**（因为都是n位数）。

◆ 为什么会有无符号类型的数？

有些数据是不用考虑负数的，比如：

- 表示**数量个数**的数据：现实世界中没有-1个之说。
- **内存地址**：内存字节的编址，从0开始，递增编址。

所以，这一类的数据就可以定义成无符号数。

好处：相比长度（位数）相同的有符号数，无符号数表示的非负数据范围大一倍。

三、数的机器码表示

1. 数的真值

记一个数 x 的绝对值为 X ，若 x 为正数，以 $+X$ 形式表示；若 x 为负数，以 $-X$ 形式表示。这种以 $\pm$ 绝对值 形式表示的数称为该数的真值。

如： $+11101$ ， -1011.0101

其中，正号可以省略，如 11101 等价于 $+11101$ ；

(这里视 x 、 X 均为二进制形式)

2. 机器数

机器数：数在计算机内部的表示称为该数的机器数。

通常，**机器数形式 $\neq$ 真值形式**

那是一种什么样的形式呢？

1) 机器数的长度

设一个二进制数 x 有 n 位，如 $x=11001$ 有5位。

由于计算机中的位 (bit) 不能单独存取，所以，一般不是直接用 n -bits表示 x ，而是用总位数不少于 n 的若干字节表示。

如： $x=(11001)_2$ ，则至少用1个字节（8bit_s）表示。

0	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---

也可用2字节（16bit_s）表示，则字节内容为：

0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

(多出来的高位填0，不影响原数的大小)

总位数=字节数×8

数的长度：一个数在计算机中表示时使用的**字节数**，称为这个**数的长度**。

注：数的长度与其**数据类型**相关，不同类型的数据长度不同，以后讲解；

2) 数在计算机中的表示：**机器数**

这里需要解决两个问题：

(1) **正负号的表示**：指定用所占字节的最高位表示，并约定最高位为0表示正数，1表示负数：



(2) **值 (大小) 的表示**：与编码方式有关(原码、反码、补码等)

机器数有原码、反码、补码等多种形式：

(1) **原码**：对于一个二进制数，令**最高位为符号位**，**其余各位为数值位**，**大小为数的绝对值**，这种**编码**称为数的**原码**。

例：十进制数**125**和**-125**的**8位**、**16位**原码表示。

$$(125)_{10} = 0X7D = (\underline{+1111101})_2$$

$$(-125)_{10} = -0X7D = (\underline{-1111101})_2$$

真值：正负号+绝对值

则根据原码的定义有：

$$\textbf{8位: } [+1111101]_{\text{原}} = \textbf{01111101} \quad [-1111101]_{\text{原}} = \textbf{11111101}$$

$$\textbf{16位: } [+1111101]_{\text{原}} = \textbf{0000000001111101}$$

$$[-1111101]_{\text{原}} = \textbf{1000000001111101}$$

◆ 原码的优点：

- 容易换算出数的真值：首位决定符号，其余位是其绝对值。

■ 原码的缺点：

- 真值0的原码表示不唯一，有+0和-0之分：

$$[+0]_{\text{原}} = 00000000 \quad (8\text{位})$$

$$[-0]_{\text{原}} = 10000000 \quad (8\text{位})$$

- 不利于减法运算的实现。

(2) 反码

反码：正数的反码与其原码相同；

负数的反码是：符号位与原码相同，其余各位是对原码的各数值位**逐位变反**所得。

取反：1变0、0变1

例：分别求十进制数6和-1在8位机中的反码表示。

$[6]_{\text{原}} = 00000110 \quad \rightarrow \quad [6]_{\text{反}} = 00000110;$

$[-1]_{\text{原}} = 10000001 \quad \quad [-1]_{\text{反}} = \mathbf{11111110}$

(3) 补码

补码：正数的补码等于其原码；

负数的补码是：符号位与原码相同，其余各位是对原码的各数值位**逐位变反**，然后整体加1。

即：负数的补码=负数的反码+1

例：求十进制数35和-1在8位机中的补码表示。

$$[35]_{\text{补}} = [35]_{\text{原}} = 00100011;$$

$$[-1]_{\text{原}} = 10000001,$$

$$[-1]_{\text{补}} = \underset{\text{-1的反码}}{11111110} + \underset{\text{-1的补码}}{1} = 11111111;$$

补码的优点

(1) 零的补码表示形式唯一：

$$[+0]_{\text{补}} = [-0]_{\text{补}} = 000...000$$

(2) 整数减法, “减一个数” 等于 “加上这个负数的补码” ,

$$\text{即 } x - y \equiv x + (-y)_{\text{补}},$$

从而将减法运算转化成补码的加法运算, 简化了加减法运算电路的实现 (具体原理以后再学) 。

3. 各种计算机编码

码：是指计算机中数据表示的抽象表述。

根据数据的类型，码可以分为**数值码、字符码和图象码**等。

- **数值码：**数值型数据的编码
- **字符码：**字符型数据的编码
- **图像码：**图像数据的编码
- **音频码：**音频数据的编码

不管是哪种数据编码，在计算机中都是用**二进制码的基本形态**存储在内存之中的。**对一个用一串0/1表示的数，怎么理解和使用取决于应用程序赋予的逻辑含义。**

四、字符的编码表示

字符数据是指英文字母、标点符号、控制字符，以及中文、日文、阿拉伯语等各民族的文字符号。

字符编码：计算机中有一个**码表**，里面对系统中的所有字符，每个都指定了一个唯一的编号，这个编号就是**字符编码**；**字符编号和字符是一一对应的。**

字库和字形：计算机中还配套的**字库**，字库里描述了每个符号的“图形”信息。显示字符时，先根据字符的编码在字库中找到字符的**图形描述**，然后再在屏幕或打印机上“画”出符号的形状。

- ◆ 字库为系统中软件共享。
- ◆ 存储程序代码时，**计算机中存储的是字符的编码。**

如：字符A的编码是67

存储字符A  存储数值67

显示字符A  用编码67查找字库，得到A的图形描述，然后在屏幕上“画”出A的图形

问：硬盘上的一个字符文件（文本文件），里面保存的实际是什么类型的数据？

答：存的是文本文件中每个字符的编码 —— **整数**，所以计算机中保存的所有数据本质上都是数值型数据。

1. 西文字符的国际标准编码——ASCII码

1967年，美国标准化组织对常用的**西文字符**（包括大、小写英文字母、阿拉伯数字0~9、英文标点符号、常用控制字符等，共128个），统一编制了一张码表，称为**ASCII码表**（American Standard Code for Information Interchange，美国国家信息交换标准字符码）。

ASCII码表中共包含**128个西文字符**，编号**0~127**。计算机中标准ASCII码采用**单字节编码**（8bit_s），且只用低**7位编码**，最高位留做校验位，默认为0。每个字符的这个编码就称为该字符的**ASCII码**。

IBM PC 的 ASCII 碼

ASCII 值 十進位	十六進位	字元 符號	ASCII 值 十進位	十六進位	字元 符號	ASCII 值 十進位	十六進位	字元 符號	ASCII 值 十進位	十六進位	字元 符號
000	00	(null)	032	20		064	40	@	096	60	
001	01	☉	033	21	!	065	41	A	097	61	a
002	02	●	034	22	"	066	42	B	098	62	b
003	03	♥	035	23	#	067	43	C	099	63	c
004	04	♦	036	24	\$	068	44	D	100	64	d
005	05	♣	037	25	%	069	45	E	101	65	e
006	06	♠	038	26	&	070	46	F	102	66	f
007	07	·	039	27	'	071	47	G	103	67	g
008	08	■	040	28	(	072	48	H	104	68	h
009	09	○	041	29	)	073	49	I	105	69	i
010	0A	■	042	2A	*	074	4A	J	106	6A	j
011	0B	♂	043	2B	+	075	4B	K	107	6B	k
012	0C	♀	044	2C	,	076	4C	L	108	6C	l
013	0D	♪	045	2D	-	077	4D	M	109	6D	m
014	0E	♪	046	2E	.	078	4E	N	110	6E	n
015	0F	*	047	2F	/	079	4F	O	111	6F	o
016	10	▶	048	30	0	080	50	P	112	70	p
017	11	◀	049	31	1	081	51	Q	113	71	q
018	12	‡	050	32	2	082	52	R	114	72	r
019	13	‡	051	33	3	083	53	S	115	73	s
020	14	¶	052	34	4	084	54	T	116	74	t
021	15	§	053	35	5	085	55	U	117	75	u
022	16	-	054	36	6	086	56	V	118	76	v
023	17	±	055	37	7	087	57	W	119	77	w
024	18	↑	056	38	8	088	58	X	120	78	x
025	19	↓	057	39	9	089	59	Y	121	79	y
026	1A	→	058	3A	:	090	5A	Z	122	7A	z
027	1B	←	059	3B	;	091	5B	[	123	7B	{
028	1C	~	060	3C	<	092	5C	\	124	7C	
029	1D	++	061	3D	=	093	5D	]	125	7D	}
030	1E	▲	062	3E	>	094	5E	^	126	7E	~
031	1F	▼	063	3F	?	095	5F	_	127	7F	■

- [1] 前32个码 (0x00到0x1f) 和最后一个码 (0x7f) 为控制码, 如回车符、换行符、振铃、制表符等;
- [2] 从0x20到0x7e共95个编码, 对应数字字符0~9, 大小写英文字母a~z和A~Z, 以及常用标点符号。

计算机的终端 (键盘、显示器) 只能输入或显示这95个编码对应的字符。打印机也只能打印输出这95个字符对应的图样。因此这95字符被称为**可打印字符**, 其余字符为不可打印字符。

ASCII码			ASCII码			ASCII码			ASCII码		
十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符	十进制	十六进制	字符
128	80	Š	160	A0	á	192	C0	Ł	224	E0	α
129	81	ü	161	A1	í	193	C1	ł	225	E1	β
130	82	é	162	A2	ó	194	C2	ŧ	226	E2	γ
131	83	â	163	A3	ú	195	C3	ţ	227	E3	π
132	84	ä	164	A4	ñ	196	C4	—	228	E4	Σ
133	85	à	165	A5	Ñ	197	C5	†	229	E5	σ
134	86	å	166	A6	ª	198	C6	†	230	E6	μ
135	87	ç	167	A7	º	199	C7	‡	231	E7	ŧ
136	88	ê	168	A8	¿	200	C8	Ł	232	E8	φ
137	89	ë	169	A9	ƒ	201	C9	Ŕ	233	E9	θ
138	8A	è	170	AA	¬	202	CA	Ŗ	234	EA	Ω
139	8B	ï	171	AB	½	203	CB	Ŧ	235	EB	δ
140	8C	î	172	AC	¼	204	CC	‡	236	EC	∞
141	8D	ì	173	AD	í	205	CD	=	237	ED	φ
142	8E	Ä	174	AE	<<	206	CE	‡	238	EE	€
143	8F	Å	175	AF	>>	207	CF	≡	239	EF	∩
144	90	É	176	B0	⋮	208	D0	⋮	240	F0	≡
145	91	æ	177	B1	⋮	209	D1	ŧ	241	F1	±
146	92	Æ	178	B2	⋮	210	D2	ŧ	242	F2	≥
147	93	ô	179	B3		211	D3	ŧ	243	F3	≤
148	94	ö	180	B4	†	212	D4	Ł	244	F4	∫
149	95	ò	181	B5	‡	213	D5	Ŕ	245	F5	∫
150	96	û	182	B6	‡	214	D6	Ŕ	246	F6	÷
151	97	ù	183	B7	ŧ	215	D7	‡	247	F7	≈
152	98	ÿ	184	B8	ŧ	216	D8	‡	248	F8	°
153	99	ÿ	185	B9	ŧ	217	D9	ŧ	249	F9	•
154	9A	Ü	186	BA		218	DA	Ŕ	250	FA	•
155	9B	Ɔ	187	BB	ŧ	219	DB	■	251	FB	√
156	9C	ƒ	188	BC	ŧ	220	DC	■	252	FC	ⁿ
157	9D	¥	189	BD	ŧ	221	DD	■	253	FD	²
158	9E	₤	190	BE	ŧ	222	DE	■	254	FE	■
159	9F	ƒ	191	BF	ŧ	223	DF	■	255	FF	

扩展ASCII码：后来IBM扩展了ASCII码表，扩展后的ASCII 码将**最高位也作为编码位**。这样总共有**256个编码**，前128个与标准ASCII码一致，后面增加了128个**字符编码**（多为一些特殊字符）。

2. 汉字编码

西文编码总共有256个，所以一字节数据可以表示。但汉字多达6万多个（常用的就有3800多），单字节无法表示。因此汉字采用**多字节编码**。

汉字编码有多种方式，常用的有：**国标码**、**区位码**、**机内码**、**输入码**等。

- 1) **输入码**：用于汉字输入的编码，如：**拼音码**和**五笔字型码**等。
- 2) **国标码**：是我国于1981年颁布的《通用汉字字符集(基本集)及其交换码标准》规定的**汉字字符编码方案**，标准编号是**GB2312-80**，总共收集了7445个汉字、字母、数字和符号。

3) 区位码

国标GD2312-80规定，所有的国标汉字及符号分配在一个**94行X94列**的方阵中，方阵的一行称为一个“**区**”，编号为01~94区；一列称为一个“**位**”，编号为01~94位（**相当于行、列坐标**）。方阵中的每个汉字和符号的“**区号**”和“**位号**”组合在一起形成的**四位阿拉伯数字码**，称为它们的“**区位码**”。

位													
(0101)	、 (0102)	° (0103)	• (0104)	~ (0105)	˘ (0106)	˙ (0107)	˚ (0108)	℥ (0109)	— (0110)				
~ (0111)	∥ (0112)	… (0113)	′ (0114)	˝ (0115)	“ (0116)	” (0117)	〔 (0118)	〕 (0119)	⟨ (0120)				
⟩ (0121)	《 (0122)	〔 (0123)	↑ (0124)	↓ (0125)	↑ (0126)	↓ (0127)	〔 (0128)	〕 (0129)	【 (0130)				
】 (0131)	± (0132)	× (0133)	÷ (0134)	∴ (0135)	∧ (0136)	∨ (0137)	Σ (0138)	Π (0139)	∪ (0140)				
∩ (0141)	∈ (0142)	∴ (0143)	√ (0144)	⊥ (0145)	// (0146)	⊥ (0147)	∧ (0148)	⊙ (0149)	f (0150)				
≠ (0151)	= (0152)	≈ (0153)	≈ (0154)	≈ (0155)	≈ (0156)	≈ (0157)	≈ (0158)	≈ (0159)	≤ (0160)				
≥ (0161)	≈ (0162)	≈ (0163)	≈ (0164)	≈ (0165)	≈ (0166)	≈ (0167)	≈ (0168)	≈ (0169)	≈ (0170)				
§ (0171)	□ (0172)	⊗ (0173)	⊗ (0174)	⊗ (0175)	§ (0176)	Na (0177)	☆ (0178)	★ (0179)	○ (0180)				
● (0181)	◎ (0182)	○ (0183)	◆ (0184)	□ (0185)	■ (0186)	△ (0187)	▲ (0188)	※ (0189)	→ (0190)				
← (0191)	↑ (0192)	↓ (0193)	≡ (0194)	i (0201)	ii (0202)	iii (0203)	iv (0204)	v (0205)	vi (0206)				
vii (0207)	viii (0208)	ix (0209)	x (0210)	(0211)	(0212)	(0213)	(0214)	(0215)	(0216)				
1. (0217)	2. (0218)	3. (0219)	4. (0220)	5. (0221)	6. (0222)	7. (0223)	8. (0224)	9. (0225)	10. (0226)				
11. (0227)	12. (0228)	13. (0229)	14. (0230)	15. (0231)	16. (0232)	17. (0233)	18. (0234)	19. (0235)	20. (0236)				

每个汉字和符号都有一个唯一的区位码，
前两位是区号，后两位是位号。

如：汉字“母”字的区位码是3624_(十进制)，
表明它在方阵的36区24位

国标码和区位码之间的换算关系：

$$\text{国标码}_{(\text{十六进制})} = \text{区位码}_{(\text{十六进制})} + 0x2020$$

如：汉字“母”字的区位码是：0x2418

3624_(十进制)

则其国标码是：0x4438

汉字编码：两字节编码

区位码、国标码都是**二字节码**，两个字节分别表示“区”和“位”，用字节的低七位进行编码（区、位编码范围都仅为94个）。因此，

- ◆ 编码长度： 16bit（两字节）
- ◆ 编码容量： $2^{16} = 65536$ 个字符
- ◆ 实际编码数： $128 * 128 = 16384$ 个字符
- ◆ 实际字符数： 7445 字符

4) 汉字的机内码

汉字机内码是计算机内部存储和处理汉字时采用的汉字编码。

编码方法：将**国标码**两个字节的**每个字节的最高位置“1”**
后形成（主要是为了与基本ASCII相互区分）。

机内码与国标码之间的转换关系是：

$$\text{机内码 (十六进制)} = \text{国标码 (十六进制)} + 0x8080$$

例如：“中”字的机内码是 0xd0d6

国标码是 0x5056

区位码是 0x3036

